



Ingeniería en Sistemas Computacionales

Asignación 1

Estudiante: Alejandro Rodriguez

Cédula: 8-988-1735

Programación de Computadoras IV

Profesor: Abdel Martínez

Base de Datos Relacional en SQLite

Gremio de Aventureros: misiones, héroes y monstruos

1. Objetivo

Diseñar una base de datos relacional en SQLite que registre las misiones realizadas por un gremio de aventureros, los héroes que participan en ellas y los monstruos enfrentados en cada misión.

2. Modelo lógico

El modelo tiene tres tablas de entidades (heroes, misiones, monstruos) y dos tablas puente que resuelven las relaciones muchos-a-muchos (misiones_heroes y misiones_monstruos).

Tabla: heroes

Información de cada héroe del gremio.

Columna	Tipo	Restricciones / notas
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
nombre	TEXT	NOT NULL
clase	TEXT	NOT NULL, CHECK en lista de clases (Guerrero, Mago, Arquero, Clérigo, Pícaro, Paladín)
nivel_experiencia	INTEGER	NOT NULL, DEFAULT 1, CHECK >= 1

Tabla: misiones

Detalles de cada misión.

Columna	Tipo	Restricciones / notas
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
nombre	TEXT	NOT NULL
descripcion	TEXT	Opcional
dificultad	INTEGER	NOT NULL, CHECK entre 1 y 10
localizacion	TEXT	NOT NULL
recompensa_oro	INTEGER	NOT NULL, DEFAULT 0, CHECK >= 0 (monedas de oro)

Tabla: monstruos

Información de cada monstruo.

Columna	Tipo	Restricciones / notas
id	INTEGER	PRIMARY KEY, AUTOINCREMENT

Columna	Tipo	Restricciones / notas
nombre	TEXT	NOT NULL
tipo	TEXT	NOT NULL, CHECK en lista de tipos (Dragón, Goblin, No-muerto, Demonio, Bestia, Gigante)
nivel_amenaza	INTEGER	NOT NULL, CHECK entre 1 y 10

Tabla: misiones_heroes

Tabla puente: participación de héroes en misiones (N:M).

Columna	Tipo	Restricciones / notas
mision_id	INTEGER	NOT NULL, FK → misiones(id), ON DELETE CASCADE
heroe_id	INTEGER	NOT NULL, FK → heroes(id), ON DELETE CASCADE
(mision_id, heroe_id)	—	PRIMARY KEY compuesta

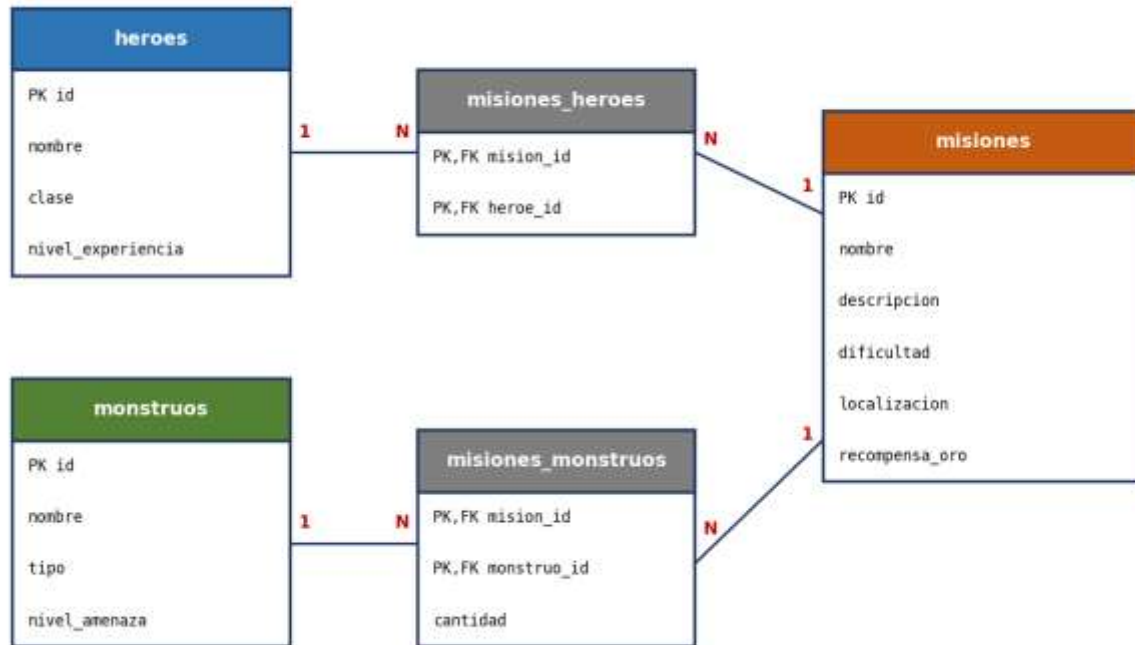
Tabla: misiones_monstruos

Tabla puente: monstruos enfrentados en cada misión (N:M).

Columna	Tipo	Restricciones / notas
mision_id	INTEGER	NOT NULL, FK → misiones(id), ON DELETE CASCADE
monstruo_id	INTEGER	NOT NULL, FK → monstruos(id), ON DELETE RESTRICT
cantidad	INTEGER	NOT NULL, DEFAULT 1, CHECK >= 1
(mision_id, monstruo_id)	—	PRIMARY KEY compuesta

3. Modelo entidad-relación (ER)

Modelo Entidad-Relación: Gremio de Aventureros



Relaciones: Héroe–Misión es N:M y Misión–Monstruo es N:M. Cada una se descompone en dos relaciones 1:N hacia su tabla puente. **cantidad** es un atributo de la relación misión–monstruo.

4. Script SQL (SQLite)

```
PRAGMA foreign_keys = ON;

CREATE TABLE heroes (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre        TEXT    NOT NULL,
    clase         TEXT    NOT NULL
    CHECK (clase IN ('Guerrero', 'Mago', 'Arquero', 'Clérigo', 'Pícaro', 'Paladín')),
    nivel_experiencia INTEGER NOT NULL DEFAULT 1
    CHECK (nivel_experiencia >= 1)
);

CREATE TABLE misiones (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre        TEXT    NOT NULL,
    descripcion    TEXT,
    dificultad     INTEGER NOT NULL
    CHECK (dificultad BETWEEN 1 AND 10),
    localizacion  TEXT    NOT NULL,
    recompensa_oro INTEGER NOT NULL DEFAULT 0
    CHECK (recompensa_oro >= 0)
);

CREATE TABLE monstruos (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre        TEXT    NOT NULL,
```

```

    tipo          TEXT      NOT NULL
    CHECK (tipo IN ('Dragón','Goblin','No-muerto','Demonio','Bestia','Gigante')),
    nivel_amenaza INTEGER NOT NULL
    CHECK (nivel_amenaza BETWEEN 1 AND 10)
);

CREATE TABLE misiones_heroes (
    mision_id INTEGER NOT NULL,
    hero_id   INTEGER NOT NULL,
    PRIMARY KEY (mision_id, hero_id),
    FOREIGN KEY (mision_id) REFERENCES misiones(id) ON DELETE CASCADE,
    FOREIGN KEY (hero_id)   REFERENCES heroes(id)   ON DELETE CASCADE
);

CREATE TABLE misiones_monstruos (
    mision_id   INTEGER NOT NULL,
    monstruo_id INTEGER NOT NULL,
    cantidad    INTEGER NOT NULL DEFAULT 1
    CHECK (cantidad >= 1),
    PRIMARY KEY (mision_id, monstruo_id),
    FOREIGN KEY (mision_id) REFERENCES misiones(id) ON DELETE CASCADE,
    FOREIGN KEY (monstruo_id) REFERENCES monstruos(id) ON DELETE RESTRICT
);

CREATE INDEX idx_mh_heroe    ON misiones_heroes(hero_id);
CREATE INDEX idx_mm_monstruo ON misiones_monstruos(monstruo_id);

```

5. Decisiones de diseño

- **PK compuesta en las tablas puente:** el par (mision_id, hero_id) impide que un héroe se registre dos veces en la misma misión.
- **PRAGMA foreign_keys = ON:** SQLite tiene las claves foráneas desactivadas por defecto; sin este pragma las cláusulas FOREIGN KEY no se aplican. Debe activarse en cada conexión.
- **CASCADE vs RESTRICT:** al borrar una misión se eliminan sus participaciones; al borrar un monstruo usado en misiones se bloquea, para conservar el historial.
- **Columna cantidad:** pertenece a la relación (por ejemplo, 5 goblins en una misión), no a la misión ni al monstruo, por eso vive en la tabla puente.
- **Índices secundarios:** la PK compuesta solo acelera búsquedas por mision_id; los índices sobre hero_id y monstruo_id cubren la dirección inversa en los JOIN.
- **Restricciones CHECK:** controlan rangos de dificultad y amenaza (1 a 10), recompensa no negativa y listas cerradas de clase y tipo.

6. Consulta de prueba

Misiones de cada héroe con los monstruos enfrentados:

```
SELECT h.nombre AS heroe, m.nombre AS mision,
       mo.nombre AS monstruo, mm.cantidad
FROM heroes h
JOIN misiones_heroes mh ON mh.heroe_id = h.id
JOIN misiones m ON m.id = mh.mision_id
JOIN misiones_monstruos mm ON mm.mision_id = m.id
JOIN monstruos mo ON mo.id = mm.monstruo_id
ORDER BY h.nombre, m.nombre;
```